

PROJECT REPORT

PYTHON BLACKJACK GAME

Submitted By

ATHARVA JOSHI

Subject: Programming in Python

Course Code: CSE1021

Academic Year: 2026–2027

Date: 29-09-2026

A console-based Blackjack game demonstrating Python fundamentals, random number generation, functions, loops, conditional statements, input validation, list operations, and game-state management.

TABLE OF CONTENTS

S. No.	Topic	Page
1	Introduction	3
2	Problem Statement	4
3	Functional Requirements	5
4	Non-Functional Requirements	6
5	Design and Game Flow	7
6	Design Decisions & Rationale	9
7	Implementation Details	10
8	Screenshots / Results	13
9	Testing Approach	15
10	Challenges Faced	16
11	Learning & Key Takeaways	17
12	Future Enhancements	18
13	Conclusion	19
14	References	20

1. Introduction

The Python Blackjack Game is a command-line application developed to demonstrate core Python programming concepts through an interactive card game. The player begins with 500 chips, places a bet, receives cards, and plays against a computer-controlled dealer.

The program uses Python's built-in random module to generate cards. Functions are used to calculate hand totals, display cards, and draw random cards. Lists are used to represent the player and dealer hands, while loops control the repeated rounds and hit/stand decisions.

The project also demonstrates an important Blackjack rule: an Ace can initially count as 11, but can be reduced to 1 when the hand would otherwise exceed 21. The dealer follows a simple automatic strategy and continues drawing until the total reaches at least 17.

Major Python Concepts Demonstrated

- Variables and data types
- Functions and parameters
- Lists and list methods
- Conditional statements
- while loops
- User input and validation
- Dictionary-based card display
- Random number generation
- Modular program design

2. Problem Statement

Develop a command-line Blackjack game in Python in which a player competes against a computer-controlled dealer using a virtual chip balance.

The program must solve the following requirements:

1. Start the player with a fixed chip balance of 500.
2. Allow the player to enter a valid bet within the available chip balance.
3. Deal two random cards to the player and dealer.
4. Display the player's hand and only the dealer's visible card during the decision stage.
5. Allow the player to choose between hitting and standing.
6. Calculate hand totals correctly, including flexible Ace values.
7. Detect a player bust when the total exceeds 21.
8. Make the dealer draw until reaching at least 17.
9. Compare the final totals and update the chip balance.
10. Continue playing until the player quits or has no chips remaining.

The project is intended as an academic programming exercise rather than a complete casino simulation. Advanced Blackjack features such as splitting, insurance, doubling down, and multiple decks are outside the scope of the current implementation.

3. Functional Requirements

1. Betting

The program asks the player for a bet and accepts only a positive integer that does not exceed the current chip balance. The player may enter 'q' to quit.

2. Card Generation

The draw() function generates a random card value from 1 to 13. Values 11, 12, and 13 represent J, Q, and K when displayed.

3. Hand Calculation

The total() function calculates the value of a hand. Number cards retain their value, face cards count as 10, and an Ace initially counts as 11. If the total is above 21, Aces are converted from 11 to 1 as required.

4. Player Decision

While the player's total is below 21, the program accepts 'h' for hit or another response for stand. A hit adds another random card to the player's list.

5. Dealer Decision

After the player stands, the dealer automatically draws cards while the dealer total is below 17.

6. Result and Chip Update

The program compares the player and dealer totals. A player win increases chips by the bet, a loss decreases chips by the bet, and an equal score results in a push with no chip change.

4. Non-Functional Requirements

Usability

The game should provide clear prompts such as the current chip balance, available betting range, hand totals, and hit/stand choices.

Reliability

The program should correctly calculate hand totals and update the chip balance without allowing bets greater than the available balance.

Performance

The application uses simple calculations and small lists, so game operations execute immediately for a normal command-line session.

Maintainability

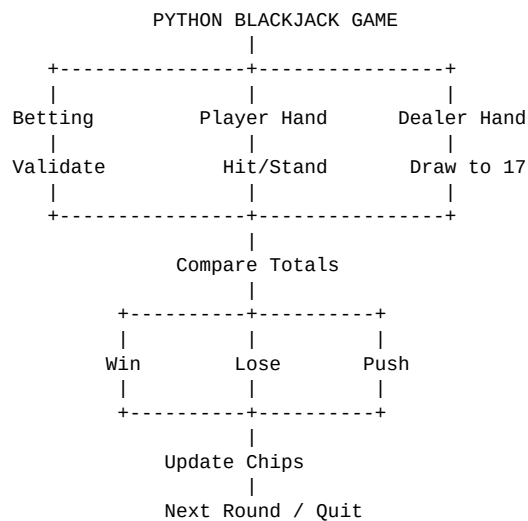
The program separates repeated operations into functions such as `total()`, `show()`, and `draw()`, making the code easier to understand and modify.

Portability

Only the Python standard library is required, so the program can run on systems with Python 3.x installed.

5. Design and Game Flow

Program Structure



Game Workflow

START → Initialize chips = 500 → Ask for bet → Validate bet → Deal two cards → Display player hand and dealer's visible card → Player chooses hit/stand → Check for bust → Dealer draws until 17 → Compare scores → Update chips → Repeat while chips > 0.

Main Data Structures

player: list containing the player's cards.

dealer: list containing the dealer's cards.

chips: integer representing the player's current balance.

bet: integer representing the wager for the current round.

names: dictionary mapping Ace and face-card values to display symbols.

6. Design Decisions & Rationale

1. Python

Python was selected because it provides concise syntax and built-in support for lists, dictionaries, functions, loops, input handling, and random number generation.

2. Console-Based Interface

A console interface keeps the project focused on programming logic rather than GUI development. It also makes the game easy to run and test from a terminal.

3. Functions

The functions `total()`, `show()`, and `draw()` isolate repeated operations and make the main game loop easier to read.

4. Lists for Hands

Player and dealer cards are stored in lists because a hand can change size when additional cards are drawn.

5. Flexible Ace Handling

The `total()` function first treats Aces as 11 and then reduces their value by 10 when the hand exceeds 21. This provides the standard flexible Ace behavior used by Blackjack.

6. Simple Dealer Strategy

The dealer draws while the total is below 17. This provides a clear deterministic rule suitable for a beginner programming project.

7. Implementation Details

Card Total Calculation

```
def total(hand):
    t = sum(min(c, 10) if c > 1 else 11 for c in hand)
    aces = hand.count(1)

    while t > 21 and aces:
        t -= 10
        aces -= 1

    return t
```

The expression assigns 11 to an Ace and limits all other cards to a maximum of 10. If the resulting score is greater than 21, each available Ace reduces the total by 10, effectively changing that Ace from 11 to 1.

Card Display

```
def show(hand):
    names = {1: "A", 11: "J", 12: "Q", 13: "K"}
    return " ".join(names.get(c, str(c)) for c in hand)
```

Random Card Generation

```
def draw():
    return random.randint(1, 13)
```

Bet Validation

```
if not bet.isdigit() or not 1 <= int(bet) <= chips:
    print("Invalid bet.")
    continue
```

8. Complete Source Code

The following is the Python implementation used for the project.

```
import random

def total(hand):
    t = sum(min(c, 10) if c > 10 else c for c in hand)
    aces = hand.count(1)

    while t > 21 and aces:
        t -= 10
        aces -= 1
    return t

def show(hand):
    names = {1: "A", 11: "J", 12: "Q", 13: "K"}
    return " ".join(names.get(c, str(c)) for c in hand)

def draw():
    return random.randint(1, 13)

chips = 500

while chips > 0:
    bet = input(f"Chips: {chips}. Bet (or q to quit): ")
    if bet == "q":
        break
    if not bet.isdigit() or not 1 <= int(bet) <= chips:
        print("Invalid bet.")
        continue
    bet = int(bet)

    player = [draw(), draw()]
    dealer = [draw(), draw()]

    while total(player) < 21:
        print(f"Your hand: {show(player)} = {total(player)} | Dealer shows: {show([dealer[1]])}")
        choice = input("(h)it or (s)and? ")
        if choice == "h":
            player.append(draw())
        else:
            break

    if total(player) > 21:
        print(f"Bust with {total(player)}! You lose {bet}.")
        chips -= bet
        continue

    while total(dealer) < 17:
        dealer.append(draw())

    p, d = total(player), total(dealer)
    print(f"You: {show(player)} = {p} | Dealer: {show(dealer)} = {d}")

    if d > 21 or p > d:
        print(f"You win {bet}!")
        chips += bet
    elif p < d:
        print(f"You lose {bet}.")
        chips -= bet
    else:
        print("Push.")

print(f"You leave with {chips} chips.")
```

9. Screenshots / Results

Figure 1: Console execution of the Python Blackjack Game

```
Chips: 500. Bet (or q to quit): 499
Your hand: 7 A = 18 | Dealer shows: J
(h)it or (s)tand? s
You: 7 A = 18 | Dealer: 6 J 7 = 23
You win 499!
Chips: 999. Bet (or q to quit): 999
Your hand: A 4 = 15 | Dealer shows: 8
(h)it or (s)tand? h
You: A 4 6 = 21 | Dealer: 8 8 Q = 26
You win 999!
Chips: 1998. Bet (or q to quit): 1998
Your hand: 2 Q = 12 | Dealer shows: J
(h)it or (s)tand? h
Your hand: 2 Q 5 = 17 | Dealer shows: J
(h)it or (s)tand? h
Bust with 27! You lose 1998.
You leave with 0 chips.
PS C:\Users\VICTUS\OneDrive\Desktop\project vit>
```

The screenshot shows multiple rounds of execution. It demonstrates betting, card display, hit/stand input, dealer card generation, winning outcomes, a bust, chip updates, and the final state when the player reaches zero chips.

Observed Execution

- Starting balance: 500 chips.
- A bet of 499 chips resulted in a player win and increased the balance to 999.
- A second round with a 999-chip bet resulted in another player win, increasing the balance to 1998.
- A third round ended with a player bust at 27, causing a loss of 1998 chips.
- Final displayed balance: 0 chips.

10. Testing Approach

The game was tested using functional testing with different bets, card combinations, player decisions, and end-game conditions.

Test ID	Test Case	Input / Condition	Expected Result
TC01	Valid bet	Bet within chip balance	Round starts
TC02	Invalid bet	Bet greater than balance	Invalid bet message
TC03	Quit	Enter q	Game exits
TC04	Hit	Enter h	A new card is added
TC05	Stand	Enter s	Player turn ends
TC06	Player bust	Total > 21	Bet is lost
TC07	Dealer draw	Dealer total < 17	Dealer draws
TC08	Player win	Player > dealer	Bet is added
TC09	Dealer win	Dealer > player	Bet is deducted
TC10	Push	Equal totals	Chip balance unchanged

The provided execution screenshot confirms that the implemented program performs the main game cycle successfully, including repeated rounds, chip updates, wins, and bust handling.

11. Challenges Faced

1. Correct Ace Calculation

Aces can have two values in Blackjack. Handling an Ace as 11 first and converting it to 1 when necessary requires additional logic.

2. Bet Validation

The program must prevent invalid bets, including non-numeric input, zero, negative values, and bets greater than the available chips.

3. Managing Repeated Rounds

The outer while loop must correctly continue the game after each round while also stopping when the player quits or has no chips.

4. Dealer Logic

The dealer must automatically draw cards without asking the user for a decision. The program therefore uses a separate loop that continues until the dealer reaches 17 or more.

5. Randomness During Testing

Because cards are generated randomly, different executions can produce different results. Testing therefore requires observing multiple rounds and different possible outcomes.

12. Learning & Key Takeaways

Technical Learnings

- Writing reusable Python functions
- Working with lists
- Using dictionaries for mappings
- Generating random values
- Applying conditional logic
- Using while loops for repeated processes
- Validating user input
- Managing program state with variables

Practical Learnings

The project demonstrates that even a small game requires planning of rules, data representation, input handling, decision logic, testing, and clear output. Breaking the program into functions makes the implementation easier to understand and maintain.

Key Takeaway

The most important learning is that programming is not only about writing statements. A complete program requires logical decomposition, validation, testing, and careful management of the application's state.

13. Future Enhancements

The current project can be expanded in several ways:

1. Full Blackjack Rules

Add natural Blackjack detection, doubling down, splitting, insurance, and surrender.

2. Multiple Decks

Use an actual shuffled deck or multiple decks instead of independently generating values from 1 to 13.

3. Graphical User Interface

Develop a GUI using Tkinter, PyQt, or another Python GUI framework.

4. Score and Statistics

Track total games, wins, losses, pushes, win percentage, and highest chip balance.

5. Persistent Player Profile

Store player statistics and chip balances so that game progress can be restored later.

6. Improved Input Handling

Accept uppercase and lowercase commands consistently and provide clearer prompts for invalid choices.

7. Automated Testing

Create unit tests for the hand-total calculation, betting validation, and result logic.

14. Conclusion

The Python Blackjack Game successfully demonstrates the use of fundamental Python programming techniques in an interactive application. The program implements betting, random card generation, hand scoring, flexible Ace handling, player decisions, dealer logic, result calculation, and chip management.

The execution shown in the supplied screenshot demonstrates multiple successful rounds, including wins and a bust, followed by the player's chip balance reaching zero. The project therefore provides a practical example of how basic programming concepts can be combined to build a complete command-line application.

15. References

1. Python Software Foundation — Python 3 Documentation.
2. Python random module documentation — random number generation.
3. Course/project guidelines for the CSE1021 programming project.
4. Supplied project-report template/structure used as the basis for this report.

Student: Atharva Joshi

Project: Python Blackjack Game

Course Code: CSE1021